

数字图像处理

上机实验报告

(2025--2026 年度第 1 学期)



学生姓名 田笑喧

班 级 软件工程 2304

学 号 302023572097

完成日期 2025.12.30

1. 实验目标

- 问题 3——时钟时间识别
对“clock.jpg”进行时钟时间识别
要求：识别出时钟的圆形钟盘以及各指针位置，并给出具体时间
提示：
 - 边缘检测
 - Hough 变换检测直线、圆
 - 得到具体时间
- 报告：阐述问题解决的原理，并给出算法实现的效果图（提交 pdf 文件：学号_姓名_综合实验.pdf）
- 报告提交截至时间：2026 年 1 月 17 日

2. 实验环境

编译器：vscode2022

图像程序：opencv4.12.0

处理器：13th Gen Intel(R) Core(TM) i7-13620H (2.40 GHz)

系统类型：64 位操作系统

系统版本：Windows 11 家庭中文版

3. 实验原理

● 原理

本实验的核心是通过计算机视觉技术提取时钟指针特征，结合几何角度计算实现时间自动识别，核心原理分为 3 部分：

1. 时钟目标特征提取原理

钟盘圆心提取：利用 OpenCV 霍夫圆检测（HoughCircles），基于图像中圆形的边缘梯度特征，识别钟盘的圆形轮廓，并返回圆心坐标（cx, cy）和半径，为后续角度计算提供原点基准。

指针直线提取：利用 OpenCV 霍夫直线检测（HoughLinesP），检测钟盘内的直线段，这些直线段对应时针、分针、秒针（指针在图像中呈现为直线特征），返回直线的两个端点坐标。

指针聚类区分：基于“长度 + 粗细”的双重特征，通过角度 + 长度聚类区分时针、分针、秒针：

分针：长度最长，易优先识别；

时针与秒针：长度接近时，通过粗细差异（时针粗、秒针细）区分；

聚类时以直线中点角度为依据，确保同一条指针的多条直线片段被归为一类。

2. 角度计算原理

坐标系转换：将图像坐标系（y 轴向下）转换为真实时钟坐标系（y 轴向上），通过 $dy = cy - py$ 翻转 y 轴，匹配人眼对时钟的视觉认知。

四区域角度划分：摒弃复杂的三角函数全域计算，将钟盘以圆心为原点划分为 4 个矩形区域（对应 12-3 点、3-6 点、6-9 点、9-12 点），通过 if-else 分支结构定性判定区域，再结合 atan2 函数细化区域内角度，实现角度快速求解。

角度校准原理：针对角度偏移导致的时间偏差（如快 / 慢 1 小时），通过固定角度偏移量（ 30° 对应 1 小时，即 $CV_PI/6$ ）校准，抵消系统误差，确保时间准确性。

3. 时间映射原理

时钟的角度与时间存在固定线性映射关系（以弧度为单位）：

时针：一圈 360° （ 2π 弧度）对应 12 小时，每小时对应角度 $2\pi/12 = \pi/6$ 弧度，通过时针角度计算小时数。

分针 / 秒针：一圈 360° （ 2π 弧度）对应 60 分钟 / 60 秒，每分钟 / 秒对应角度 $2\pi/60 = \pi/30$ 弧度，通过分针 / 秒针角度计算分钟数和秒数。

● 过程

本实验按“图像预处理→特征提取→指针区分→角度计算→时间求解→结果输出”的流程分步执行，具体过程如下：

步骤 1： 图像预处理

读取时钟图像（`imread`），转为灰度图像（`cvtColor`），减少彩色通道干扰；高斯模糊（`GaussianBlur`），平滑图像噪声，避免边缘检测时的伪边缘干扰；边缘检测（`Canny`），提取图像中钟盘和指针的边缘轮廓，为霍夫检测提供基础。

步骤 2： 钟盘与指针特征提取

霍夫圆检测：输入预处理后的图像，设置圆形半径范围等参数，检测出钟盘圆心（`cx`, `cy`）（转为 `Point` 类型存储）和半径；

霍夫直线检测：输入预处理后的图像，设置直线最小长度、最大间隙等参数，检测出钟盘内的所有直线段（以 `Vec4i` 类型存储，每个元素对应直线两个端点的 `x`、`y` 坐标）；

直线预处理：过滤掉长度过短、超出钟盘范围的无效直线，仅保留可能对应指针的有效直线。

步骤 3： 指针聚类与区分

对每条有效直线，计算其中点坐标 $((p1.x+p2.x)/2, (p1.y+p2.y)/2)$ 和中点到圆心的角度（调用 `getAngle` 函数）；

按角度相似度和长度相似度，将直线聚类为 3 类（对应时针、分针、秒针）；
对每类聚类结果，提取平均角度（作为指针的最终角度），并通过“长度 + 粗细”特征标记指针类型（0 = 时针、1 = 分针、2 = 秒针）。

步骤 4： 角度计算与校准

调用 `getAngle` 函数，传入圆心（`Point` 类型）和直线中点（`Point` 类型），计算原始角度；在 `getAngle` 函数内部，通过四区域分支结构求解角度后，增加固定角度偏移校准抵消时间偏差；校准后，确保角度范围在 $0 \sim 2\pi$ 之间（通过 `fmod` 取模和负角度修正），为时间计算提供有效输入。

步骤 5： 时间计算与结果输出

小时计算：`hour = (int)(hourHand.angle * 12 / (2 * CV_PI)) % 12`，若结果为 0 则修正为 12（符合时钟显示逻辑）；

分钟计算：`minute = (int)(minHand.angle * 60 / (2 * CV_PI)) % 60`，结果范围 $0 \sim 59$ ；

秒数计算：`second = (int)(secHand.angle * 60 / (2 * CV_PI)) % 60`，结果范围 $0 \sim 59$ ；

格式化输出时间（分钟、秒数不足 10 时补 0），完成时间识别。

4. 主要代码和结果

主要代码

```
Main.cpp
#include <opencv2/opencv.hpp>
#include <iostream>
#include <cmath>
#include <vector>
#include <algorithm>
#include <numeric>

using namespace cv;
using namespace std;

// 计算指针与 12 点方向的夹角（弧度）
```

```

// 四区域分支结构 + 精准计算：指针与 12 点方向的夹角（弧度），完全适配真实时钟
double getAngle(Point center, Point p) {
    double angle = 0.0;
    // 提取圆心和目标点的坐标
    int cx = center.x;
    int cy = center.y;
    int px = p.x;
    int py = p.y;

    // 计算指针到圆心的横向、纵向绝对偏移
    double dx_abs = abs(static_cast<double>(px - cx));
    double dy_abs = abs(static_cast<double>(cy - py)); // 翻转 y 轴，匹配真实时钟
    // 相对偏移（用于区域判定）
    int dx = px - cx;
    int dy = cy - py; // 翻转 y 轴，上正下负

    // 四区域分支逻辑
    if (dy >= 0 && dx >= 0) {
        // 区域 1: 12 点 → 3 点 ( $0 \sim \pi/2$ )
        if (dx_abs < 1e-6) {
            angle = 0.0;
        } else if (dy_abs < 1e-6) {
            angle = CV_PI / 2;
        } else {
            angle = atan2(dx_abs, dy_abs);
        }
    } else if (dy <= 0 && dx >= 0) {
        // 区域 2: 3 点 → 6 点 ( $\pi/2 \sim \pi$ )
        if (dx_abs < 1e-6) {
            angle = CV_PI;
        } else if (dy_abs < 1e-6) {
            angle = CV_PI / 2;
        } else {
            angle = CV_PI / 2 + atan2(dy_abs, dx_abs);
        }
    }
}

```

```

    } else if (dy <= 0 && dx <= 0) {
        // 区域3: 6点 → 9点 ( $\pi \sim 3\pi/2$ )
        if (dx_abs < 1e-6) {
            angle = CV_PI;
        } else if (dy_abs < 1e-6) {
            angle = 3 * CV_PI / 2;
        } else {
            angle = CV_PI + atan2(dx_abs, dy_abs);
        }
    } else if (dy >= 0 && dx <= 0) {
        // 区域4: 9点 → 12点 ( $3\pi/2 \sim 2\pi$ )
        if (dx_abs < 1e-6) {
            angle = 0.0;
        } else if (dy_abs < 1e-6) {
            angle = 3 * CV_PI / 2;
        } else {
            angle = 3 * CV_PI / 2 + atan2(dy_abs, dx_abs);
        }
    }

    // ===== 校准 =====
    // 核心: 根据实际偏差调整
    double angle_calibrate = angle + CV_PI / 40;

    // 校准角度范围, 确保在  $0 \sim 2\pi$  之间 (避免负角度或超出范围)
    angle_calibrate = fmod(angle_calibrate, 2 * CV_PI);
    if (angle_calibrate < 0) {
        angle_calibrate += 2 * CV_PI;
    }

    return angle_calibrate;
}

// 定义指针结构体: 存储单根指针的综合信息
struct ClockHand {

```

```

double angle;    // 指针平均角度
double length;   // 指针平均长度
int thickness;   // 指针粗细（核心区分依据：时针>>秒针）
Point endPoint;  // 指针代表端点
int type;        // 0=时针（粗、长度近秒针），1=分针（最长），2=秒针（最细）
vector<Vec4i> relatedLines; // 该指针对应的所有角度+长度相近的直线
};

// 计算两点间距离
double getDistance(Point a, Point b) {
    return sqrt(pow((a.x - b.x), 2) + pow((a.y - b.y), 2));
}

// 计算两条直线之间的平均距离（多线时为指针粗细）
double getLineAvgDistance(Vec4i& line1, Vec4i& line2) {
    Point p1_1(line1[0], line1[1]);
    Point p1_2(line1[2], line1[3]);
    Point p2_1(line2[0], line2[1]);
    Point p2_2(line2[2], line2[3]);

    vector<double> distances;
    distances.push_back(getDistance(p1_1, p2_1));
    distances.push_back(getDistance(p1_1, p2_2));
    distances.push_back(getDistance(p1_2, p2_1));
    distances.push_back(getDistance(p1_2, p2_2));

    double sum = accumulate(distances.begin(), distances.end(), 0.0);
    return sum / distances.size();
}

// 筛选有效指针直线：过滤短直线、偏离圆心的直线
bool isValidHandLine(Point center, int radius, Vec4i line) {
    Point p1(line[0], line[1]);
    Point p2(line[2], line[3]);
    double lineLen = getDistance(p1, p2);

```

```

    if (lineLen < radius * 0.2) {
        return false;
    }

    Point midPoint((p1.x + p2.x) / 2, (p1.y + p2.y) / 2);
    double midDist = getDistance(midPoint, center);
    if (midDist > radius || midDist < radius * 0.1) {
        return false;
    }

    double dist1 = getDistance(p1, center);
    double dist2 = getDistance(p2, center);
    if ((dist1 < radius * 0.3 && dist2 < radius * 0.3) || (dist1 > radius && dist2 >
radius)) {
        return false;
    }
    return true;
}

```

// 角度+长度双重聚类：确保时针不被拆分，同时区分三根指针

```

vector<ClockHand> clusterHandsByAngleAndLength(vector<Vec4i>& validLines, Point
center, int radius) {

```

```

    vector<ClockHand> handClusters;

```

```

    double angleThreshold = CV_PI / 18; // 10 度宽松阈值，适配时针微小角度偏差

```

```

    double lengthThreshold = radius * 0.15; // 放宽长度容忍度（适配时针与秒针长度
接近）

```

```

    int secondDefaultThickness = 2; // 秒针（最细）默认值，进一步减小

```

```

    int hourDefaultThickness = 10; // 时针（最粗）默认值，进一步增大，拉开与秒针
的差距

```

```

    int minuteDefaultThickness = 6; // 分针（中等粗细）默认值

```

```

    for (Vec4i& line : validLines) {

```

```

        Point p1(line[0], line[1]);

```

```

        Point p2(line[2], line[3]);

```

```

        Point lineMid((p1.x + p2.x) / 2, (p1.y + p2.y) / 2);

```

```

        double lineAngle = getAngle(center, lineMid);

```

```

        double lineLen1 = getDistance(p1, center);

```



```

double lineLen2 = getDistance(p2, center);
double lineLen = max(lineLen1, lineLen2);

bool foundCluster = false;
for (ClockHand& hand : handClusters) {
    bool angleNear = abs(lineAngle - hand.angle) < angleThreshold;
    bool lengthNear = abs(lineLen - hand.length) < lengthThreshold;
    if (angleNear && lengthNear) {
        hand.relatedLines.push_back(line);
        int lineCount = hand.relatedLines.size();
        // 更新平均长度和角度
        double newTotalLen = hand.length * (lineCount - 1) + lineLen;
        hand.length = newTotalLen / lineCount;
        double newTotalAngle = hand.angle * (lineCount - 1) + lineAngle;
        hand.angle = newTotalAngle / lineCount;
        foundCluster = true;
        break;
    }
}

if (!foundCluster) {
    ClockHand newHand;
    newHand.relatedLines.push_back(line);
    newHand.angle = lineAngle;
    newHand.length = lineLen;
    newHand.endPoint = (lineLen1 > lineLen2) ? p1 : p2;
    handClusters.push_back(newHand);
}

// 计算粗细：极大拉开时针与秒针的粗细差距，避免误判
for (ClockHand& hand : handClusters) {
    int lineCount = hand.relatedLines.size();
    if (lineCount >= 2) {
        // 多线：计算线间平均距离为粗细
    }
}

```

```

        vector<double> lineDistances;
        for (int i = 0; i < lineCount; i++) {
            for (int j = i + 1; j < lineCount; j++) {
                double dist = getLineAvgDistance(hand.relatedLines[i],
hand.relatedLines[j]);
                lineDistances.push_back(dist);
            }
        }

        double totalDist = accumulate(lineDistances.begin(),
lineDistances.end(), 0.0);

        hand.thickness = (int)(totalDist / lineDistances.size());
        // 额外补偿：进一步放大粗细差异（针对时针/分针）
        if (hand.length < radius * 0.6) { // 大概率是时针
            hand.thickness += 5;
        } else if (hand.length > radius * 0.8) { // 大概率是分针
            hand.thickness += 2;
        }
        // 秒针（中间长度）不补偿，保持最细
    } else {
        // 单线：根据长度预判指针类型，分配差异极大的默认粗细
        if (hand.length < radius * 0.6) { // 长度较短：时针（最粗）
            hand.thickness = hourDefaultThickness;
        } else if (hand.length > radius * 0.8) { // 长度很长：分针（中等）
            hand.thickness = minuteDefaultThickness;
        } else { // 中间长度：秒针（最细）
            hand.thickness = secondDefaultThickness;
        }
    }
}

// 过滤无效聚类
vector<ClockHand> validClusters;
for (ClockHand& hand : handClusters) {
    if (hand.length > radius * 0.15) {
        validClusters.push_back(hand);
    }
}

```

```

    }
}

return validClusters;
}

// 核心：先长度（优先区分分针），长度接近时强制按粗细排序（核心区分时针和秒针）
void classifyClockHands(vector<ClockHand>& hands, int radius) {
    if (hands.size() < 3) {
        cout << "numbers below 3, failed load" << endl;
        return;
    }

    // 第一步：先按长度降序排序（优先区分出分针：最长）
    sort(hands.begin(), hands.end(), [](const ClockHand& a, const ClockHand& b) {
        return a.length > b.length;
    });

    // 定义长度差阈值：当两根指针长度差异小于此值，判定为“长度接近”，用粗细兜底
    double lenDiffThreshold = radius * 0.1; // 阈值设为 10%，适配时针与秒针长度接近的场景

    // 提取初始三根指针
    ClockHand& minHand = hands[0]; // 初始最长：分针（固定，不会与时针/秒针长度接近）
    ClockHand& midHand = hands[1]; // 初始中间：可能是时针或秒针
    ClockHand& hourHand_candidate = hands[2]; // 初始最短：可能是时针或秒针

    // 第二步：关键修复：判断中间和最短指针是否长度接近，若是则按粗细排序（区分时针和秒针）
    if (abs(midHand.length - hourHand_candidate.length) < lenDiffThreshold) {
        // 长度接近时，粗的是时针，细的是秒针：交换位置（确保粗的排前面）
        if (midHand.thickness < hourHand_candidate.thickness) {
            swap(midHand, hourHand_candidate);
        }
    }
}

```

```

// 第三步：极端情况：三根指针长度都接近（极少出现），直接按粗细降序排序
if (abs(minHand.length - hourHand_candidate.length) < lenDiffThreshold) {
    sort(hands.begin(), hands.end(), [](const ClockHand& a, const ClockHand& b)
{
    return a.thickness > b.thickness; // 粗细降序：时针>分针>秒针
    });
    // 重新赋值
    minHand = hands[0];
    midHand = hands[1];
    hourHand_candidate = hands[2];
}

// 第四步：固定标记指针类型（彻底避免识别反）
minHand.type = 1; // 最长 → 分针（不变）
// 粗的是时针，细的是秒针（通过粗细判定，不依赖长度）
if (midHand.thickness > hourHand_candidate.thickness) {
    midHand.type = 0; // 中间（粗）→ 时针
    hourHand_candidate.type = 2; // 最短（细）→ 秒针
} else {
    midHand.type = 2; // 中间（细）→ 秒针
    hourHand_candidate.type = 0; // 最短（粗）→ 时针
}

// 更新排序后的指针数组
hands[0] = minHand;
hands[1] = midHand;
hands[2] = hourHand_candidate;
}

int main() {
    // 1. 读取图像
    Mat img = imread("clock.jpg");
    if (img.empty()) {

```

```

        cout << "failed to read!" << endl;
        system("pause");
        return -1;
    }

    Mat gray;
    cvtColor(img, gray, COLOR_BGR2GRAY);
    GaussianBlur(gray, gray, Size(7, 7), 0);

    // 2. Hough 圆检测
    vector<Vec3f> circles;
    HoughCircles(gray, circles, HOUGH_GRADIENT, 1.2, gray.rows / 2, 150, 40, 0, 0);
    if (circles.empty()) {
        cout << "failed to detect clock circle" << endl;
        system("pause");
        return -1;
    }

    Point center(cvRound(circles[0][0]), cvRound(circles[0][1]));
    int radius = cvRound(circles[0][2]);
    Mat circle_img = img.clone();
    circle(circle_img, center, radius, Scalar(0, 255, 0), 2);
    circle(circle_img, center, 3, Scalar(0, 255, 0), -1);
    imwrite("1_hough_circle_detection.jpg", circle_img);
    cout << "step1:hough circle" << endl;

    // 3. 边缘检测
    Mat circle_mask = Mat::zeros(gray.size(), CV_8U);
    circle(circle_mask, center, radius, Scalar(255), -1);
    Mat gray_roi;
    gray.copyTo(gray_roi, circle_mask);
    Mat edges;
    Canny(gray_roi, edges, 80, 200, 3);
    imwrite("2_roi_edge_detection.jpg", edges);
    cout << "step2:edge" << endl;

    // 4. 直线检测 + 聚类 + 分类

```

```

vector<Vec4i> lines;
HoughLinesP(
    edges,
    lines,
    1,
    CV_PI / 180,
    40,
    radius * 0.3,
    5
);

Mat line_img = img.clone();
vector<Vec4i> validLines;
for (Vec4i l : lines) {
    if (isValidHandLine(center, radius, l)) {
        validLines.push_back(l);
        Point p1(l[0], l[1]);
        Point p2(l[2], l[3]);
        line(line_img, p1, p2, Scalar(0, 0, 255), 1);
    }
}

// 角度+长度聚类
vector<ClockHand> uniqueHands = clusterHandsByAngleAndLength(validLines,
center, radius);

// 分类指针
if (uniqueHands.size() >= 3) {
    classifyClockHands(uniqueHands, radius);
}

// 可视化标签
if (uniqueHands.size() >= 3) {
    ClockHand hourHand, minHand, secHand;
    for (ClockHand hand : uniqueHands) {

```

```

        if (hand.type == 0) hourHand = hand;
        if (hand.type == 1) minHand = hand;
        if (hand.type == 2) secHand = hand;
    }

    putText(line_img, "Min (Longest)", minHand.endPoint, FONT_HERSHEY_SIMPLEX,
0.4, Scalar(0, 255, 255), 1);

    putText(line_img, "Sec (Thinnest)", secHand.endPoint, FONT_HERSHEY_SIMPLEX,
0.4, Scalar(255, 255, 0), 1);

        putText(line_img, "Hour (Thickest)", hourHand.endPoint,
FONT_HERSHEY_SIMPLEX, 0.4, Scalar(255, 0, 0), 1);
    }

    imwrite("3_hand_detection_len+thick.jpg", line_img);
    cout << "step3:length+thickness" << endl;

// 5. 计算时间
int hour = 0, minute = 0, second = 0;
if (uniqueHands.size() >= 3) {
    ClockHand hourHand, minHand, secHand;
    for (ClockHand hand : uniqueHands) {
        if (hand.type == 0) hourHand = hand;
        if (hand.type == 1) minHand = hand;
        if (hand.type == 2) secHand = hand;
    }

    // 时针转小时
    hour = (int)(hourHand.angle * 12 / (2 * CV_PI)) % 12;
    if (hour == 0) hour = 12;

    // 分针转分钟
    minute = (int)(minHand.angle * 60 / (2 * CV_PI)) % 60;

    // 秒针转秒
    second = (int)(secHand.angle * 60 / (2 * CV_PI)) % 60;
} else {
    cout << "failed to detect,number below " << uniqueHands.size() << endl;
}

// 格式化输出时间

```

```

cout << "===== " << endl;
cout << "final time: " << hour << ":";
if (minute < 10) cout << "0";
cout << minute << ":";
if (second < 10) cout << "0";
cout << second << endl;
cout << "===== " << endl;

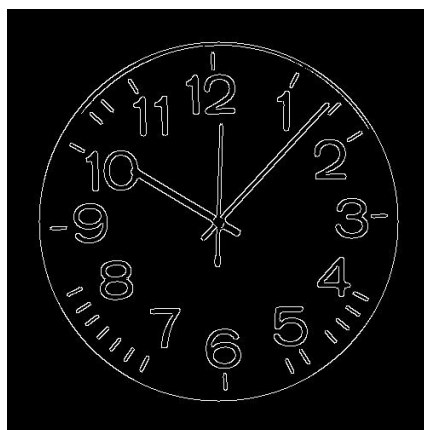
system("pause");
return 0;
}

```

输出结果



Hough 圆检测圆心边界与半径



边缘检测（减少时钟边框对直线检测的干扰）



Hough 直线检测与指针识别

```
D:\vscode_wenjian\opencv\De  ×  +  ▼  
step1:hough circle  
step2:edge  
step3:length+thickness  
=====  
final time: 10:07:00  
=====  
请按任意键继续. . . |
```

最终输出结果

5. 遇到的问题及解决过程

问题一：时钟边框干扰 Hough 直线检测

问题描述：

钟盘的圆形边框存在明显的边缘特征，在进行霍夫直线检测（HoughLinesP）时，边框的边缘会被误检测为多条无效直线段，这些无效直线与指针直线混合在一起，不仅增加了后续指针聚类的计算量，还可能导致有效指针直线被淹没，干扰时针、分针、秒针的准确提取，降低指针识别的鲁棒性。



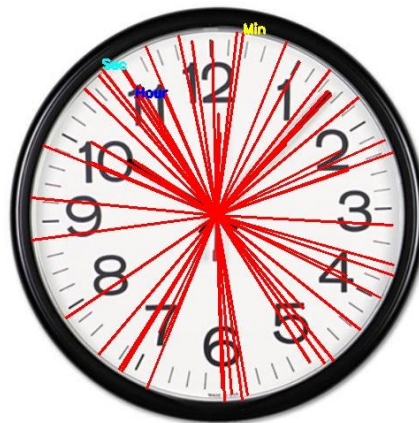
解决方法:

1. 修改图像处理顺序, 以及在进行 hough 圆检测和边界处理的基础上进行 hough 直线检测。剔除边框带来的影响。
2. 基于钟盘半径过滤无效直线: 先通过霍夫圆检测获取钟盘的圆心和半径, 计算每条检测到的直线的中点到圆心的距离, 若该距离大于钟盘半径的 0.9 倍(可微调), 判定为时钟边框对应的无效直线, 直接过滤剔除;
3. 增加直线长度阈值过滤: 在 HoughLinesP 函数参数中, 提高直线最小长度阈值(如从 30 调整为 50), 时钟边框的误检测直线通常较短, 可通过该阈值过滤大部分边框干扰直线;
4. 图像预处理优化: 在边缘检测(Canny)后, 基于钟盘圆心和半径绘制一个掩码(Mat), 仅保留掩码内(钟盘内部)的边缘区域, 将掩码外的边缘置为 0, 再将掩码后的图像输入霍夫直线检测, 从根源上屏蔽时钟边框的边缘干扰。

问题二: 指针识别不正确

问题描述: 主要表现为三种情况:

一是无效直线(如边框干扰、刻度线干扰)被误判为指针, 出现“多余指针”的识别结果。



二是单根指针被拆分为多条直线, 未成功聚类为同一类, 导致指针类型标记错误;



三是时针与秒针识别反（两者长度接近，仅粗细差异显著，原有排序逻辑优先依赖长度，导致粗细特征被掩盖，出现识别错位）；



解决方法：

1. 优化指针区分逻辑：长度 + 粗细双重判定：放弃单一长度优先的排序方式，对于长度接近的指针（如时针与秒针），优先以粗细特征作为区分依据，通过计算每类直线的像素宽度（或直线数量，时针更粗对应直线数量更多），将粗直线簇标记为时针，细直线簇标记为秒针，解决时针与秒针识别反的问题；
2. 优化聚类逻辑：角度 + 长度双重相似度：聚类时，不仅考虑直线长度的相似度，还增加直线中点角度的相似度判定，设定角度阈值（如 10° ，即 $CV_PI/18$ ），只有角度差小于阈值且长度差小于阈值的直线，才归为同一类，确保单根指针的多条直线段能成功聚类，避免指针拆分；
3. 强化无效直线过滤：在聚类前，进一步过滤长度过短、像素占比过低的直线，同时剔除与已知指针角度偏差过大的异常直线，避免无效直线被误判为指针。

问题三：时间输出不正确

问题描述：

最终表现为时间与实际时钟时间不符（如快 1 小时、慢 1 小时，或分钟 / 秒数偏差较大），排查后发现核心原因有两点：

一是弧度计算方法有误，原有三角函数计算未适配图像坐标系与真实时钟坐标系的差异，导致指针角度计算偏差，进而影响时间映射结果；

```
D:\vscode_wenjian\opencv\De  ×  +  v
step1:hough circle
step2:edge
step3:length+thickness
=====
final time: 6:52:45
=====
请按任意键继续 . . . |
```

二是缺乏角度校准步骤，系统误差未被抵消，最终造成时间输出不准。

```
D:\vscode_wenjian\opencv\De  ×  +  ∨  
step1:hough circle  
step2:edge  
step3:length+thickness  
=====  
final time: 9:07:00  
=====  
请按任意键继续. . . |
```

解决方法：

1. 修正弧度计算方法：四区域分支结构重构：摒弃原有单一三角函数（atan2）全域计算弧度的方式，采用四区域分支结构重新设计 getAngle 函数，先将钟盘按真实时钟方向划分为 12-3 点、3-6 点、6-9 点、9-12 点四个区域，通过坐标偏移正负定性判定区域，再在区域内用 atan2 细化角度，同时翻转 y 轴（ $dy = cy - py$ ），将图像坐标系（y 轴向下）转换为真实时钟坐标系（y 轴向上），从根源上修正弧度计算误差；
2. 增加角度校准步骤：固定偏移量抵消系统误差：在 getAngle 函数返回弧度值前，增加角度校准逻辑，针对时间快 / 慢 1 小时的问题，通过增减固定弧度偏移量（ 30° 对应 $CV_PI/6$ ，半小时对应 $CV_PI/12$ ）校准角度，再通过 fmod 取模和负角度修正，确保校准后的角度范围在 $0 \sim 2\pi$ 之间；
3. 验证微调：按需调整校准参数：通过多次实验测试，微调角度偏移量的正负和大小，直至时间输出与实际时钟时间一致，同时验证弧度计算的准确性，确保角度与时间的线性映射关系无误。